



# SDK for music streaming



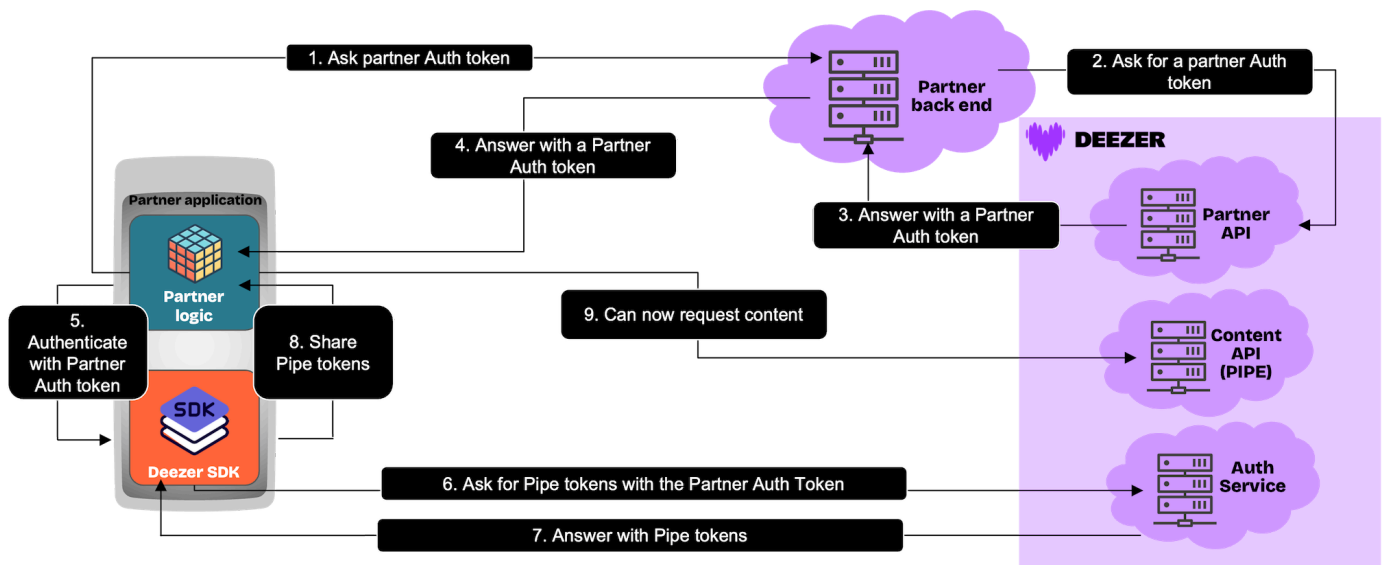
**Deezer SDK** addresses 3 use cases to enable music streaming your app:

- **User authentication** (verify the identity of a user, provide a proof of authentication, refresh the proof)
- **Player and queuelist management** (Play Deezer's music (DRM, Media fetching...), provide basic playback controls, queue list management)
- **Download management** (give offline access to Deezer content)

## SDK functional overview

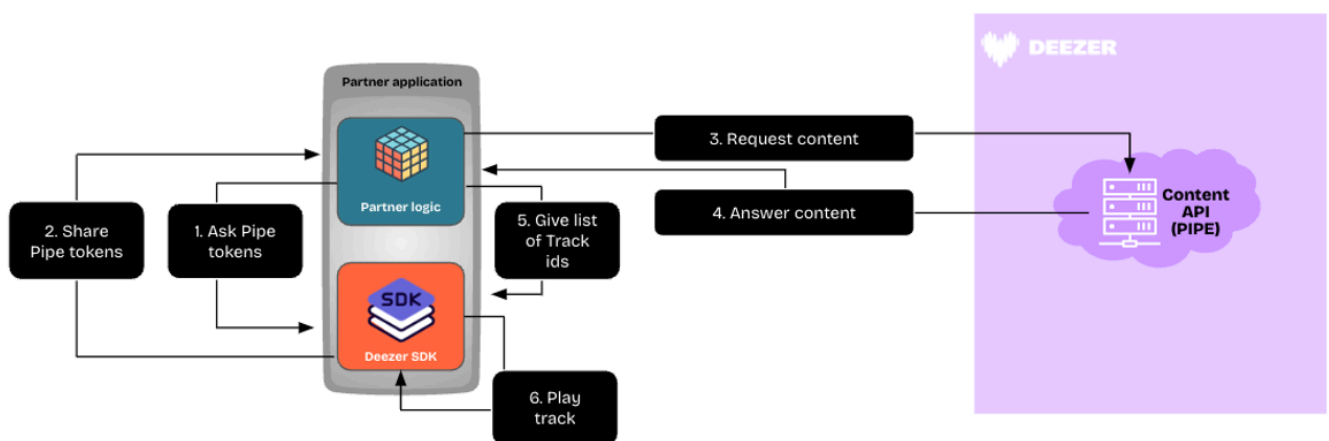
### How to access content for B2B users

You can use the SDK to retrieve Pipe authentication token for its user to let them access to Deezer content



## How partner could play music

You can use the SDK to play Deezer music, add new tracks to the queue list and provide basic playback controls



## List of features available in the player

### Player actions

- Play
- Pause
- Previous
- Next
- Seek

- goTo

## QueueList

- SetQueue()
  - Infinite (Flow)
  - Limited (list of tracks ID)
  - Track
- Add
- Remove
- Current Index
- Current Media
- Shuffle
- Repeat mode (All/One/None)
- isEditable (infinite queue list cannot be edited)

## Player information

- Playback state (Buffering/Playing/...)
- Playback errors
- SIP
- Duration
- Current position in the track

## Limitations

## Compatibility

As of December 2023, our SDKs are compatible with the following versions:

- iOS > Minimum version: 13
- Android > Minimum version of API level: 23

Android

iOS

**Web**

---

# How to use the SDK

## Private registry setup

You need to use npm to fetch the two necessary packages `@deezer/sdk-auth` and `@deezer/sdk-player`

To configure your machine to download '@deezer' packages from the right registry, run the following command

### Setup custom registry

```
npm config set @deezer:registry  
https://deezerpps.jfrog.io/artifactory/api/npm/web-sdk/
```

### Login to the registry

Then, use your JFrog credentials to login to the repository by running (you'll be asked to provide the credentials detailed below):

```
npm login --scope=@deezer --  
registry=https://deezerpps.jfrog.io/artifactory/api/npm/web-sdk/
```

JFrog credentials are provided to you by Deezer, check on the RC documentation on Confluence to find them

### Try to install deezer packages

You should now be able to install the deezer SDK packages in your application by running:

```
npm i @deezer/sdk-auth @deezer/sdk-player
```

## Authentication via the SDK Web Auth API

### Setup & rules to follow

- You should only use a single instance of the SDK Auth across your app
- We recommend that you create your instance at the top of the app and expose the instance to the rest of the app
- The SDK instance must be created only after the user has logged in and has valid tokens

Note: For instance, we made a React context provider to create the SDKAuth instance and expose it to our components

```
const authInstance = new Auth({
  host: "https://auth.deezer.com",
  customFetch,
});
```

## Configuration

The Auth class can be configured with the following parameters:

| param name             |                               |          | description                                                                                                                                     |
|------------------------|-------------------------------|----------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| host                   | string                        | required | The host of the authentication service                                                                                                          |
| customFetch            | typeof<br>globalThis.fetch    | optional | A custom fetch method if need to decorate requests (e.g. with metrics). Defaults to <code>fetch</code>                                          |
| tokenOutputType        | QUERY_PARAMS_TYPE<br>(c or p) | optional | Type of output for the JWT Defaults to <code>QUERY_PARAMS_TYPE.PAYLOAD</code>                                                                   |
| refreshTokenOutputType | QUERY_PARAMS_TYPE<br>(c or p) | optional | Type of output for the Refresh Token Defaults to <code>QUERY_PARAMS_TYPE.COOKIE</code>                                                          |
| requestDelay           | number                        | optional | Delay (in seconds) to invalidate a token before its real expiration date to ensure validity throughout the request. Defaults to <code>15</code> |

**Note:** We **strongly** recommend to use the default configuration which should be adapted to most usage.

If you need to override the default behavior, be sure to know what you are doing and please arise any issue that might occur as some config matching might not be fully viable.

## API

Once you have instantiated an Auth class, you can use the following methods:

| method name            | signature                     | description                                                                                                                                          |
|------------------------|-------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| getToken               | '() ⇒ Promise<string   null>' | Returns a JWT fresh from auth service or from cache if still valid. Returns <code>null</code> if <code>tokenOutputType</code> is <code>COOKIE</code> |
| invalidateCurrentToken | '() ⇒ void'                   | Forces the invalidation of tokens cached into the Auth class                                                                                         |
| logout                 | '() ⇒ Promise<void/>'         | Invalidates JWT and refresh token set in cache or cookie                                                                                             |

## Login as Partner

```
await authInstance.loginAsPartner(partnerToken);
```

If login is successfull, a refresh\_token will be stored as a cookie. This refresh\_token will be used by the auth module to get a fresh JWT anytime it's invalidated.

Once logged in, calling `authInstance.getToken()` will automatically handle any use of the refresh\_token and should always give you a valid JWT.

`authInstance.logout()` can be used to remove the cookies and reset the auth instance.

`authInstance.invalidateCurrentToken` can be used if a service using the JWT (the Pipe GraphQL API in our case) tells you that the JWT is invalid. This will force the next use of `getToken` to use the refresh\_token to get a new valid JWT.

## Alternative storage for refresh tokens

You might be experiencing issues with the token renewal process with some browsers (e.g. Safari). This is due to browsers's default setting of preventing cross-site tracking. A workaround for this is to use an alternative storage:

```
import Auth, {
  Storage,
  LocalStorageRefreshTokenStorage,
  QUERY_PARAMS_TYPE,
} from "@deezer/sdk-auth";

const storage = new Storage();
const authInstance = new Auth({
  host: "auth.deezer.com",
  refreshTokenOutputType: QUERY_PARAMS_TYPE.PAYLOAD,
  refreshTokenCache: storage.isStorageSupported
    ? new LocalStorageRefreshTokenStorage(storage)
    : undefined,
});
```

## Player

### WARNING

If you want to change the default configuration of the player use

`DeezerSdk.configurePlayer()` before any access to the player. It will be executed once upon first access

The player is divided into 4 parts

- [actions](#)
- [playback information](#)
- [queue list management](#)
- [settings](#)

### Actions

The player provides a comprehensive API for controlling playback behavior, such as speed, volume, navigation, and seeking.

### Playback Speed Control

- `setPlaybackSpeed`
  - Set the current playback speed multiplier.
  - Values in parameter (number between 0 and 5).

- Sends an action to the player to change the speed.
- **getPlaybackSpeed**
  - Retrieve the current playback speed multiplier.
  - If drive not available, return 1.0 (normal speed) by default.
  - A read-only method that doesn't modify anything.

### **DANGER**

Playback speed modification is **not allowed** for music media

## Volume Control

- **volume**
  - Represents the playback volume as a fraction of the system media volume.
  - Acceptable range is `[0.0, 1.0]`, with out-of-range values coerced to the nearest limit.

## Playback Actions

- **play()**
  - Starts or resumes playback of the current content.
- **pause()**
  - Pauses playback of the current content.
- **previous()**
  - Skips to the previous content in the queue.
- **next()**
  - Skips to the next content in the queue.

## Seeking and Jumping

- **seek(positionInSeconds)**
  - Seeks to a specific position in the current content, specified in seconds from the start.
- **goTo(index, startPositionInSeconds)**
  - Jumps to a specific media content in the queue based on its `index`.
  - Starts playback at the specified `startPositionInSeconds`.

- If the `index` is invalid or not authorized for the current content, the jump is not executed.

## Playback Information

The player provides a set of methods for interacting with the playback system. It allows developers to retrieve playback information, subscribe to various playback events, and manage subscription callbacks for seamless integration with the Deezer ecosystem.

### TIP

- **Callback Reference Management:** Callbacks and listeners must be retained explicitly to prevent them from being garbage collected.
- **Event Subscription and Unsubscription:** For every `subscribe` method, there is a corresponding `unsubscribe` method to manage event listeners effectively.

## Playback State Management

- `getPlaybackState()`
  - Retrieves the current playback state.
- `subscribeOnPlaybackStateChanged(callback)`
  - Subscribes to changes in the playback state.
- `unsubscribeOnPlaybackStateChanged(callback)`
  - Unsubscribes from playback state changes.

## Playback Progress

- `subscribeOnPlaybackProgress(callback)`
  - Subscribes to playback progress updates.
- `unsubscribeOnPlaybackProgress(callback)`
  - Unsubscribes from playback progress updates.

## Error Handling

- `subscribeOnPlayerError(callback)`
  - Subscribes to errors occurring in the Deezer Player.
- `unsubscribeOnPlayerError(callback)`
  - Unsubscribes from player error notifications.

## Media Change Events

- `subscribeOnCurrentMediaChanged(callback)`
  - Subscribes to changes in the current media being played.
- `unsubscribeOnCurrentMediaChanged(callback)`
  - Unsubscribes from current media change notifications.

## Playback Limitations

- `subscribeOnPlayerLimitation(listener)`
  - Subscribes to events related to playback limitations (e.g., restricted content).
- `unsubscribeOnPlayerLimitation(listener)`
  - Unsubscribes from playback limitation events.

## Playback Information

- `getBufferedPosition()`
  - Retrieves the position up to which media content is buffered, in seconds.
- `getCurrentPosition()`
  - Retrieves the current playback position, in seconds.
  - Related method: `subscribeOnPlaybackProgress`.
- `getDuration()`
  - Retrieves the total duration of the currently playing media, in seconds.

## Queue list management

The player provides a comprehensive API for managing and interacting with a playback queue. It supports asynchronous operations for modifying the queue, retrieving playback-related data, and subscribing to playback state changes.

### TIP

- **Callback Reference Management:** Callbacks must be retained explicitly to prevent unexpected garbage collection.
- **Asynchronous Operations:** All queue modification methods are asynchronous, with optional callbacks to handle results or errors.

## Queue Management

- **setQueue(playable, callback, index)**
  - Sets a new queue asynchronously, replacing any existing queue.
  - Starts playback from the specified index (or index `0` if null or out of bounds).
  - Callback returns the updated queue or a `QueueException` on failure.
- **add(playable, callback, index)**
  - Adds playable content to the current queue asynchronously.
  - Inserts at the specified index (or appends to the end if null or out of bounds).
  - Callback returns the updated queue or a `QueueException` on failure.
- **removeAt(index, callback)**
  - Removes a track at the specified index asynchronously.
  - Callback returns the updated queue or a `QueueException` on failure.
- **canEdit()**
  - Indicates if the queue allows editing (e.g., adding or removing items).
- **subscribeOnQueueUpdated(callback)**
  - Subscribes to queue updates.
- **unsubscribeOnQueueUpdated(callback)**
  - Unsubscribes from queue updates.

## Playback Information

- **getCurrentIndex()**
  - Retrieves the current playback position's index in the queue.
- **getCurrentMedia()**
  - Returns details of the currently playing media.
- **getAllMedia()**
  - Provides a list of all tracks in the queue.

## Shuffle Management

- **canShuffle()**
  - Indicates if shuffle playback is supported.
- **isShuffled()**
  - Indicates if shuffle playback is currently enabled.
- **setShuffle(isShuffled)**
  - Enables or disables shuffle playback.

- **subscribeOnShuffleStateChanged(callback)**
  - Subscribes to shuffle state changes.
- **unsubscribeOnShuffleStateChanged(callback)**
  - Unsubscribes from shuffle state changes.

### Repeat Mode Management

- **getAllowedRepeatModes()**
  - Retrieves the allowed repeat modes for the queue.
- **getRepeatMode()**
  - Gets the current repeat mode.
- **setRepeatMode(repeatMode)**
  - Updates the repeat mode for the queue.
- **subscribeOnRepeatModeChanged(callback)**
  - Subscribes to repeat mode changes.
- **unsubscribeOnRepeatModeChanged(callback)**
  - Unsubscribes from repeat mode changes.

#### NOTE

Find more details on how to use the queue list [here](#)

### Settings

The player provides methods and properties for managing and customizing playback and storage settings, including cache size, audio quality, and volume normalization.

### Media Cache Management

#### INFO

Methods not used for the web version.

### Audio Quality

- **audioQuality**
  - The current audio quality used for both playback and downloads.

- **updateAudioQuality(quality)**
  - Updates the audio quality for playback and downloads.
  - Returns the actual audio quality set, which may differ from the requested value if unsupported by the player.

#### **Volume Normalization**

- **isVolumeNormalizationEnabled**
  - A property to enable or disable volume normalization.
  - Volume normalization ensures consistent sound levels across different tracks.